

STŘEDOŠKOLSKÁ ODBORNÁ ČINNOST
Obor č. 18: Informatika

GenAnalyzer – bioinformatický
nástroj pro porovnání
proteinových sekvencí a studium
genetických variant

Yasmin Fazla
Zlínský kraj

Zlín 2025

STŘEDOŠKOLSKÁ ODBORNÁ ČINNOST
Obor č. 18: Informatika

GenAnalyzer – bioinformatický
nástroj pro porovnání
proteinových sekvencí a studium
genetických variant

GenAnalyzer – bioinformatics tool
for protein sequence comparison
and genetic variant analysis

Autor: Yasmin Fazla

Škola: Gymnázium a Jazyková škola s právem státní jazykové
zkoušky Zlín, nám. T. G Masaryka 2734, 760 01 Zlín

Kraj: Zlínský kraj

Konzultant: Ing. Ivana Burgetová, Ph.D.

Zlín 2025

Prohlášení

Prohlašuji, že jsem svou práci SOČ vypracovala samostatně a použila jsem pouze prameny a literaturu uvedené v seznamu bibliografických záznamů.

Prohlašuji, že tištěná verze a elektronická verze soutěžní práce SOČ jsou shodné.

Nemám závažný důvod proti zpřístupňování této práce v souladu se zákonem č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) ve znění pozdějších předpisů.

Ve Zlíně dne 27. 2. 2025 Yasmin Fazla

Poděkování

Ráda bych poděkovala paní Ing. Ivaně Burgetové, Ph.D., za přínosné schůzky, během kterých jsem se měla možnost na cokoliv zeptat, za cenné rady a trpělivost při vedení mé práce.

Dále bych chtěla vyjádřit své díky panu učiteli Mgr. Michalu Miklášovi, bez jehož motivace a povzbuzení bych se na Středoškolskou odbornou činnost určitě nedala.

Anotace

Tato práce se zabývá vývojem webové aplikace, jejímž cílem je usnadnit analýzu proteinových sekvencí, identifikaci mutací a zkoumání jejich souvislostí s genetickými chorobami. Aplikace sjednocuje data z různých zdrojů a umožňuje uživatelům snadné porovnání sekvencí s referenčními variantami. Součástí řešení je otevřený zdrojový kód.

Klíčová slova

webová aplikace; analýza proteinových sekvencí; mutace; genetické choroby

Annotation

This document describes a web application designed to facilitate the analysis of protein sequences, the identification of mutations, and the exploration of their links to genetic diseases. The application consolidates data from various sources and allows users to easily compare sequences with reference variants. The solution includes open-source code.

Keywords

web application; protein sequence analysis; mutations; genetic diseases

Obsah

1	Úvod	6
2	Základy genetických mutací	7
2.1	Genetická informace	7
2.1.1	Genetický kód	7
2.1.2	Proteosyntéza	7
2.2	Genové mutace	8
2.2.1	Typy genových mutací	9
2.2.2	Důsledky genových mutací	9
2.2.3	Biologický význam mutací	10
3	Data	11
3.1	Databáze	11
3.1.1	UniProt	11
3.1.2	ClinVar	11
3.2	Formáty dat	12
3.2.1	FASTA	12
3.2.2	HGVS	12
4	Použité technologie	14
4.1	Python	14
4.1.1	Biopython	14
4.2	Flask	15
4.2.1	Routing	15
4.2.2	Blueprints	15
4.2.3	Správa session	16
4.3	JSON	16
4.4	HTML, CSS	17
4.5	JavaScript	17
4.5.1	D3.js	18
4.6	WSGI server	19
5	Implementace	20
5.1	Vyhledání v databázi UniProt	20
5.2	Zarovnání sekvencí	22
5.3	Detekce mutací	23
5.3.1	Substituce	23
5.3.2	Delece	24
5.3.3	Inzerce	24
5.3.4	Extenze	24
5.3.5	Výstup a zobrazení výsledků	25
5.4	Hledání asociací s nemocemi	25
5.4.1	Komunikace s ClinVar API	26
5.4.2	Zobrazení výsledků	27
5.5	Advanced Sequence Aligner	27
5.5.1	Přizpůsobení parametrů zarovnání	28

5.5.2	Získání dat z nástroje Protein Analyzer	28
6	Výstup a vizualizace	31
6.1	Vizualizace zarovnání sekvencí	31
6.1.1	Funkce drawAlignment()	31
6.1.2	Funkce drawD3Alignment()	32
6.2	Zobrazení tabulek mutací a asociací	33
6.2.1	Filtrace podle typu mutace	33
6.2.2	Řazení tabulky podle pozice mutace	33
6.2.3	Filtrace podle stavu varianty	33
6.2.4	Rozšířené filtrování sloupců	34
6.2.5	Export dat	34
7	Zpětná vazba	36
8	Závěr	38
8.1	Budoucí rozvoj	38
8.1.1	Rozšíření podpory dalších databází	38
8.1.2	Podpora dalších typů sekvencí	38
8.1.3	Přihlášení uživatele	38
9	Bibliografie	39
	Seznam obrázků	42
	Seznam ukázek kódu	42
	Seznam tabulek	42
	Přílohy	43

1 Úvod

Tento dokument popisuje vývoj a funkcionalitu webové aplikace *GenAnalyzer*, kterou naleznete na adrese <https://genanalyzer.pythonanywhere.com/>. Cílem bylo vytvořit aplikaci, která usnadňuje analýzu proteinových sekvencí, identifikaci mutací a zkoumání jejich souvislostí s genetickými chorobami.

Původní motivací pro vznik tohoto projektu bylo vytvořit nástroj, který by na základě genetické informace pacienta určoval jeho predispozice k určitým onemocněním. Tento záměr se postupně rozšířil na vývoj nástroje zaměřeného na analýzu proteinových mutací, čímž se odlišuje od jiných dostupných řešení podobného typu a stává se spíše nástrojem pro výzkum genetických variací.

V současnosti existuje řada databází a nástrojů, které umožňují práci s proteinovými sekvencemi, jejich používání však často vyžaduje specializované znalosti a práci s příkazovou řádkou. *GenAnalyzer* sjednocuje data z různých zdrojů a umožňuje snadné porovnání sekvencí s referenčními variantami, čímž celý proces výrazně zjednodušuje.

Aplikace je určena vědcům, studentům i vývojářům. Výzkumníkům umožňuje efektivně vyhledávat mutace a jejich vztah k onemocněním, studentům molekulární biologie a medicíny usnadňuje práci s bioinformatickými nástroji a vývojářům poskytuje open-source řešení k dalšímu využití. Díky kombinaci intuitivního uživatelského rozhraní, propojení s důležitými databázemi a možnosti dalšího vývoje má *GenAnalyzer* potenciál stát se užitečným nástrojem pro vědeckou obec.

Video s prezentací aplikace je k dispozici na platformě YouTube: <https://youtu.be/AZq-n9UdfKI>.

2 Základy genetických mutací

Bioinformatika je obor na pomezí informatiky a biologie, který se zabývá analýzou biologických dat, zejména sekvencí nukleových kyselin a proteinů. S rozvojem sekvenčních technologií získává bioinformatika stále větší význam v oblastech, jako je genomika či personalizovaná medicína.[1]

Pro pochopení fungování aplikace *GenAnalyzer* je důležité se seznámit se základními pojmy z genetiky a molekulární biologie, zejména s principy mutací v proteinových sekvencích. Ty mohou měnit strukturu a funkci proteinů, což může vést k narušení buněčných procesů a rozvoji genetických onemocnění.

Tato kapitola poskytuje teoretický základ pro pochopení genetických mutací a jejich biologických dopadů. Zároveň objasňuje význam analýzy proteinových sekvencí, což je klíčové pro správné využití aplikace *GenAnalyzer*.

2.1 Genetická informace

Geny obsahují informace o formě a vlastnostech potomstva. Tato informace se nazývá genetická informace. Genetická informace je uložena v DNA a prostřednictvím proteosyntézy překládána do proteinů, které jsou nezbytné pro fungování živých organismů.

2.1.1 Genetický kód

Genetický kód je soubor pravidel, podle kterých se informace uložená v DNA převádí na sekvenci aminokyselin tvořících proteiny. DNA se skládá z 4 nukleových bází A, G, C a T (adenin, guanin, cytosin a thymín). [2]

Trojice nukleotidů (triplet) kóduje vždy jednu konkrétní aminokyselinu. Tyto triplety se nazývají kodony. DNA obsahuje 64 kodonů, které určují jednotlivé aminokyseliny potřebné pro syntézu proteinů.

Aminokyselina	Zkratka	DNA kodony
Alanin	Ala	GCT, GCC, GCA, GCG
Asparagin	Asn	AAT, AAC
Cystein	Cys	TGT, TGC
Glutamin	Gln	CAA, CAG
Leucin	Leu	TTA, TTG, CTT, CTC, CTA, CTG
Methionin	Met	ATG
Valin	Val	GGT, GTC, GTA, GTG

Tabulka 1: Příklady přiřazení aminokyselin k jednotlivým kodonům [3]

Jak lze vidět v tabulce, jedna aminokyselina může být kódována více různými kodony. Tento mechanismus snižuje negativní dopad mutací na proteiny.[4]

2.1.2 Proteosyntéza

Translace neboli proteosyntéza, je proces, při kterém se nukleotidová sekvence mRNA překládá do sekvence aminokyselin, čímž vzniká funkční protein. Tento proces se řídí pravidly genetického kódu.[5]

Základními stavebními jednotkami proteinů jsou aminokyseliny, kterých se v lidském těle vyskytuje 20 (některé zdroje uvádějí až 22).

Každá z těchto proteinogenních aminokyselin má jednopísmenné i třípísmenné označení. Jednopísmenné zkratky se běžně používají při zápisu aminokyselinových sekvencí proteinů, zatímco třípísmenné označení se využívá zejména při zápisu variant nebo mutací, aby se předešlo záměně s označením nukleových bází, které mají podobné symboly.[6]

Jednopísmenný symbol	Třípísmenný symbol	Aminokyselina
A	Ala	Alanin
C	Cys	Cystein
D	Asp	Kyselina asparagová
E	Glu	Kyselina glutamová
F	Phe	Fenylalanin
G	Gly	Glycin
H	His	Histidin
I	Ile	Isoleucin
K	Lys	Lysin
L	Leu	Leucin
M	Met	Methionin
N	Asn	Asparagin
P	Pro	Prolin
Q	Gln	Glutamin
R	Arg	Arginin
S	Ser	Serin
T	Thr	Threonin
V	Val	Valin
W	Trp	Tryptofan
Y	Tyr	Tyrosin

Tabulka 2: Přehled zkratk aminokyselin [6]

Pořadí aminokyselin v řetězci tvoří primární strukturu proteinu, která je klíčová pro jeho specifickou funkci.

2.2 Genové mutace

Mutace jsou trvalé a dědičné změny genetické informace. Jedná se o změny v sekvenci nukleotidů oproti normální sekvenci DNA. Mohou vznikat spontánně jako důsledek chyb při replikaci DNA nebo být indukovány vnějšími faktory, jako jsou chemické látky či UV záření.[7]

V aplikaci *GenAnalyzer* je hlavní funkcí porovnání proteinových sekvencí a detekce mutací na úrovni aminokyselin, nikoliv na úrovni DNA nebo RNA. Pro srovnání sekvencí DNA nebo RNA je k dispozici nástroj *Advanced Sequence Aligner*, který je také součástí aplikace. V této práci se proto primárně soustředím na mutace, které mění aminokyselinovou sekvenci proteinů a mají biologický význam.

2.2.1 Typy genových mutací

Mezi nejčastější genové mutace patří mutace bodové, které postihují pouze jeden nukleotid. Mezi základní typy bodových mutací patří:

- **Substituce** – záměna jedné nukleotidové báze za jinou. Důsledky této změny závisí na tom, na které pozici kodonu k substituci došlo.
- **Inzerce** – vložení jednoho nebo více nadbytečných nukleotidů do sekvence. Pokud je počet vložených nukleotidů násobkem tří ($3n$), dochází k prodloužení polypeptidového řetězce. Pokud však počet vložených nukleotidů není dělitelný třemi, dochází k posunu čtecího rámce, což může vést ke vzniku nesmyslné sekvence aminokyselin nebo předčasnému ukončení proteosyntézy.
- **Delece** – ztráta jednoho nebo více nukleotidů původní sekvence. Podobně jako u inzerce může vést k posunu čtecího rámce, což výrazně ovlivní výsledný protein. [8]

Původní sekvence: $A - T - T - G - C - T$

Substituce: $A - \textcolor{blue}{A} - T - G - C - T$

Inzerce: $A - T - T - G - C - \textcolor{green}{C} - T$

Delece: $A - T - T - G - \textcolor{red}{-} - T$

Tyto mutace se vyskytují na úrovni DNA, avšak obdobným způsobem se označují i bodové mutace v sekvenci aminokyselin.

2.2.2 Důsledky genových mutací

Mutace mohou vzniknout jak v kódujících, tak v nekódujících oblastech genomu. V nekódující oblasti se většinou neprojeví, jedná se o tzv. tiché mutace. Pokud dojde k mutaci v kódujícím úseku DNA, lze je dále dělit podle toho, jak ovlivní proteosyntézu.[9]

Dopad mutace závisí na tom, zda kodon se zaměněnou bází kóduje stejnou aminokyselinu, jinou aminokyselinu, nebo zda vzniká stop kodon:

- **Tiché mutace** (silent mutation) - Nemění funkci proteinu, protože zaměněný kodon kóduje stejnou aminokyselinu.
- **Mutace měnící smysl** (missense mutation) - Způsobí zařazení odlišné aminokyseliny při proteosyntéze. V závislosti na povaze změny může být funkce proteinu zachována, narušena nebo zcela ztracena.[8]
- **Nesmyslná mutace** (nonsense mutation) - Vede ke vzniku předčasného stop kodonu, což způsobí předčasné ukončení proteosyntézy. Výsledkem je obvykle nefunkční protein.[9]

2.2.3 Biologický význam mutací

Mutace mohou mít různé biologické důsledky v závislosti na tom, kde a jak k nim došlo:

- **Neutrální mutace** – Vzhledem k tomu, že jen malá část lidského genomu (cca 1,5 %) kóduje proteiny, dochází k většině mutací v nekódujících oblastech. Ty tedy často nemají žádný pozorovatelný efekt na organismus, není však pravidlem že ani mutace v této oblasti nemohou působit negativně.
- **Škodlivé mutace** – Závažnější projev mají mutace v kódujících oblastech. Z hlediska klinické genetiky mohou vést ke genetickým chorobám, vývojovým vadám nebo nádorovému bujení.[8]
- **Prospěšné mutace** – Z pohledu evoluce mohou být mutace naopak velmi užitečné, neboť mohou vést k adaptačním změnám, jako je rezistence bakterií na antibiotika.

I když existuje mnoho mutací, které se v genomu vyskytují bez pozorovatelného dopadu na výslednou proteosyntézu, a tedy i na organismus, aplikace *GenAnalyzer* se zaměřuje výhradně na mutace v sekvencích aminokyselin, tedy na úrovni proteinů. Tím se odlišuje od nástrojů, které analyzují změny v DNA či RNA bez ohledu na jejich vliv na proteinovou strukturu a funkci.

3 Data

Jelikož oblast genetického výzkumu rychle postupuje, bylo klíčové pracovat s nejaktuálnějšími daty. Proto jsem se rozhodla propojit aplikaci s databázemi, které poskytují podrobné a pravidelně aktualizované informace o proteinech a genetických variantách.

V této kapitole se zaměřuji na databázové zdroje, které jsem využila při analýze genových sekvencí a variant, a na datové formáty FASTA a HGVS, které slouží k standardizovanému zápisu genetických dat.

3.1 Databáze

Pro získání referenčních sekvencí proteinů a genetických variant jsem pracovala se dvěma hlavními databázemi – UniProt a ClinVar.

3.1.1 UniProt

UniProt je přední světová databáze proteinových sekvencí a jejich funkčních anotací. Obsahuje informace o struktuře proteinů, jejich biologické roli a vztahu k nemocem. Data v UniProt jsou pravidelně aktualizována a lze je získat prostřednictvím webového rozhraní i API. [10]

Ve své aplikaci využívám UniProt REST API, které umožňuje efektivní automatizované dotazování. [11] Konkrétně API slouží k:

- vyhledávání proteinů na základě názvu nebo identifikátoru,
- získávání referenčních sekvencí proteinů ve formátu FASTA,
- získávání informací o vztahu proteinu k nemocem na základě dostupných anotací.

Aplikace komunikuje s UniProt API prostřednictvím HTTP požadavků. Všechny relevantní výsledky, včetně seznamu nemocí spojených s daným proteinem, jsou vráceny ve formátu JSON. Díky tomu je UniProt cenným nástrojem pro analýzu proteinových mutací a jejich dopadu na lidské zdraví.

3.1.2 ClinVar

ClinVar je veřejně dostupná databáze, která shromažďuje klinické interpretace genetických variant a jejich vztah k nemocem. [12] V rámci své aplikace využívám ClinVar API k:

- vyhledání asociace konkrétní genetické varianty s nemocemi,
- získání klinické klasifikace mutace (např. "Pathogenic", "Benign").

Dotazy na ClinVar API jsou odesílány prostřednictvím NCBI¹ *eUtils* (*Entrez Programming Utilities*), sady serverových nástrojů pro přístup k databázovému systému *Entrez*. Tyto nástroje fungují na principu pevně daných URL požadavků

¹NCBI (*National Center for Biotechnology Information*) je americká instituce spravující biologické databáze a poskytující nástroje pro analýzu biologických dat.

a umožňují vyhledávat a získávat různé biomedicínské a genetické informace. [13]

K přístupu k *eUtils* existuje limit. Pokud uživatel nemá API klíč², může odesílat maximálně 3 požadavky za sekundu z jedné IP adresy. Po získání API klíče se tento limit zvyšuje na 10 požadavků za sekundu pro všechny požadavky využívající daný klíč. [14] To v mé aplikaci prozatím stačí, proto jsem se rozhodl API klíč implementovat a používat jej pro všechny dotazy na ClinVar API.

3.2 Formáty dat

Při práci s genetickými daty je nezbytné používat standardizované formáty, které umožňují efektivní přenos, ukládání a analýzu informací.

3.2.1 FASTA

FASTA je textový formát používaný v bioinformatice k reprezentaci sekvencí nukleotidů (DNA nebo RNA) nebo aminokyselin (proteinů). [15] Skládá se ze dvou hlavních částí:

1. **Definiční řádek** – první řádek, který začíná znakem ">", za nímž následuje popis sekvence (identifikátor).
2. **Sekvence** – samotná genetická sekvence, která následuje na dalších řádcích. Každý znak v sekvenci reprezentuje jeden nukleotid nebo aminokyselinu. Sekvence může být rozdělena do několika řádků. [16]

Příklad formátu FASTA:

```
>sp|P49768|PSN1_HUMAN Presenilin-1 OS=Homo sapiens
MTLPAPLSYFQNAQMSQEDNHLSTVRSQNDNRERQEHNDRRSLGHPEPLSNGRPQGNSR
QVVEQDEEEDLTKYGAKHVIMLFVPVTLQVGVVATIKSVSFYTRKDGQLIYTPFTE
DTETVGGQALHSILNAAIMISVIVMTILLVVLKYRCYKVIHAWLISSLLLLFFFSFI
YLGEVFKTYNVAVDYITVALLIWNFGVGMISIHKGPLRLQAYLIMISALMALVFIKY
LPEWTAWLILAVISVYDLVAVLCPKGPLRMLVETAQERNETLFPALISSTMVWLVNMAE
GDPEAQRRVSKNSKYNAESTERESQDTVAENDDGGFSEWEAQRDShLGPHRSTPESRAA
VQELSSSILAGEDPEERGVKLGDFIFYSVLVGKASATASGDWNTTIACFVAILIGLCL
TLLLLAIFKKALPALPISITFGLVFYFATDYLVPFMDQLAFHQFYI
```

V aplikaci využívám formát FASTA k:

- získání sekvencí proteinů z databáze UniProt,
- načtení sekvencí od uživatele.

3.2.2 HGVS

HGVS (*Human Genome Variation Society nomenclature*) je standardizovaný způsob zápisu genetických variant, který slouží k přesné identifikaci mutací v sekvencích DNA, RNA a proteinů.

Příklady zápisu proteinových mutací podle HGVS:

²API klíč je unikátní řetězec, který se přidává do HTTP požadavků a identifikuje uživatele na serverech NCBI.

- **Substitute:** p.A1a23Val (substituce aminokyseliny alanin na pozici 23 za valin)
- **Delece:** p.Asn20_Leu22del (delece aminokyselin na pozicích 20–22)
- **Inserce:** p.His38_Asn39insGln (vložení aminokyseliny glutaminu mezi pozice 38 a 39)
- **Extenze:** p.Met1ext-3 (přidání tří dodatečných aminokyselin před metioninem na pozici 1) [17]

V aplikaci používám formát HGVS pro označování mutací, což zajišťuje kompatibilitu s databázemi jako ClinVar a podporuje výměnu genetických informací v širším měřítku.

4 Použité technologie

Při návrhu a vývoji aplikace bylo klíčové zvolit vhodné technologie, které by umožnily efektivní zpracování genetických dat a zároveň zajistily uživatelskou přívětivost.

Na základě těchto požadavků jsem zvolila kombinaci Pythonu pro zpracování genetických dat a komunikaci s databázemi a JavaScriptu pro vizualizaci výsledků na straně klienta. Jako webový framework jsem zvolila Flask, především díky jeho jednoduchosti a efektivní správě serverové části aplikace.

Tato kombinace technologií mi umožnila vytvořit aplikaci, která je výkonná, snadno rozšiřitelná a přístupná uživatelům bez nutnosti instalace dodatečného softwaru.

4.1 Python

Python jsem zvolila jako hlavní programovací jazyk pro tento projekt především kvůli jeho široké škále knihoven určených pro práci s biologickými daty a snadné integraci s databázemi. Na rozdíl od jazyků, jako je například C#, se vývojář nemusí starat o velikost proměnných nebo přetečení paměti, což je při práci s genetickými sekvencemi velká výhoda.

V rámci tohoto projektu jsem Python využila konkrétně pro:

- komunikaci s databázemi UniProt a ClinVar,
- zarovnávání a porovnávání sekvencí,
- klasifikaci mutací.

4.1.1 Biopython

Biopython je knihovna, která poskytuje nástroje pro analýzu nukleotidových a proteinových sekvencí a pro další specializované úlohy v bioinformatice. [18]

V tomto projektu jsem využila jeden z klíčových modulů Biopythonu a to `Bio.Align` a jeho třídu `PairwiseAligner`. Tato třída umožňuje efektivní zarovnání dvou sekvencí s možností definování různých parametrů, jako jsou penalizace za mezery či skórování shod a neshod mezi sekvencemi. [18] Níže uvedená funkce 1 využívající `PairwiseAligner` umožňuje zarovnat dvě sekvence na základě předdefinovaných nebo uživatelem zadaných parametrů.

```
1 from Bio.Align import PairwiseAligner
2
3 def align_sequences(sequence1, sequence2, mode="global",
4     match_score=2, mismatch_score=-1, open_gap_score=-5,
5     extend_gap_score=-2):
6
7     aligner = PairwiseAligner()
8     aligner.mode = str(mode)
9
10    aligner.match_score = match_score
11    aligner.mismatch_score = mismatch_score
12    aligner.open_gap_score = open_gap_score
```

```

13     aligner.extend_gap_score = extend_gap_score
14
15     alignment = aligner.align(sequence1, sequence2)[0]
16     return alignment

```

Ukázka kódu 1: Zarovnání sekvencí

Tato funkce se v projektu používá ve dvou různých částech aplikace:

1. V nástroji *Protein Analyzer*, kde zarovnáva uživatelem zadanou sekvenci s referenční. V této části se používají předdefinované parametry, což umožňuje automatickou detekci mutací.³
2. V nástroji *Advanced Sequence Aligner*, kde si uživatel může parametry zarovnávání upravovat a zkoumat, jak se změny ovlivňují výsledky analýzy sekvencí.

4.2 Flask

Flask je server-side webový framework napsaný v jazyce Python, který je ideální pro vývoj malých až středně velkých webových aplikací. [19] V tomto projektu slouží jako spojení mezi uživatelským rozhraním a backendovou logikou. Zvolila jsem si jej především kvůli jeho jednoduchosti a nízkým nárokům na konfiguraci, což umožňuje rychlý vývoj bez zbytečného nastavování. [20]

4.2.1 Routing

Flask umožňuje snadné směrování (routing) požadavků pomocí dekorátorů⁴, které přiřazují konkrétní funkce k daným URL adresám. To umožňuje definovat, jak bude aplikace reagovat na požadavky uživatelů.

Tato ukázka kódu 2 definuje základní cestu, která vrátí HTML šablonu `index.html`:

```

1 @bp.route("/")
2 def home():
3     return render_template("index.html")

```

Ukázka kódu 2: Základní routing ve Flasku

4.2.2 Blueprints

Flask umožňuje rozdělení aplikace na menší moduly pomocí *Blueprints* (instance třídy `Blueprint`).

Ve své aplikaci používám *Blueprints* k organizaci jednotlivých cest (viz ukázka kódu 4). Ty se registrují ve funkci `create_app()`, aby je mohla aplikace používat (viz ukázka kódu 3).

³Předdefinované hodnoty parametrů jsem zvolila s ohledem na biologické vlastnosti sekvencí a očekávané výsledky u běžných genetických sekvencí, u nichž se primárně hledají shody a malé mutace.

⁴Dekorátor je v Pythonu speciální funkce, která přijme jako argument funkci a vrátí novou funkci, která rozšiřuje nebo modifikuje chování té původní funkce.[21]

```

1 def create_app():
2     app = Flask(__name__)
3     app.config.from_object(Config)
4     Session(app)
5
6     from app.routes import bp as routes_bp
7
8     app.register_blueprint(routes_bp)
9     return app

```

Ukázka kódu 3: Registrace Blueprintu

```

1 from flask import Blueprint, render_template
2
3 bp = Blueprint("main", __name__)
4
5 @bp.route("/")
6 def home():
7     return render_template("index.html")
8
9 @bp.route("/about")
10 def about():
11     return render_template("about.html")

```

Ukázka kódu 4: Použití Blueprintu pro organizaci jednotlivých cest aplikace

Výhodou je, že takový blueprint pak můžete využívat i v několika různých aplikacích a snadno tak dosáhnout znovupoužitelnosti. [22]

4.2.3 Správa session

Flask umožňuje správu uživatelských session (relací), což je důležité pro uchovávání dat mezi jednotlivými požadavky. Session je bezpečně uložena na straně serveru a je identifikována pomocí unikátního identifikátoru.[23]

```

1 # store data in session
2 session["alignment"] = {
3     "aligned_reference_sequence": aligned_reference_sequence,
4     "aligned_input_sequence": aligned_input_sequence,
5 }
6 # get data from session
7 alignment_data = session.get("alignment")

```

Ukázka kódu 5: Použití session

V aplikaci jsou ukládány do session například zarovnané sekvence, aby se nemusely opakovaně posílat mezi frontendem a backendem.

4.3 JSON

JSON (*JavaScript Object Notation*) je datový formát, který slouží k přenosu dat mezi klientem a serverem. Je snadno strojově analyzovatelný a zároveň čitelný i pro člověka. Díky těmto vlastnostem je vhodný pro výměnu dat.

Pro tento formát jsem se rozhodla, protože je nezávislý na konkrétním programovacím jazyce a využívá zvyklosti jazyků rodiny C. Mezi ně patří nejen JavaScript, ale i Python, který používám v mém projektu.[24]

V tomto projektu JSON využívám také pro výměnu dat mezi aplikací a externími databázemi, například při komunikaci s ClinVar API. V následující ukázce kódu 6 funkce odešle požadavek na ClinVar API s parametrem `retmode=json`, což zajistí, že odpověď bude ve formátu JSON. Python ji převede na slovník pomocí `response.json()` a extrahuje klíčové informace, následně je vrací zpět ve formátu JSON.

```
1 def fetch_clinvar_variant_info(variant_id):
2     summary_url = f"{CLINVAR_URL}summary.fcgi"
3     params = {"db": "clinvar", "id": variant_id, "retmode": "json"}
4     try:
5         response = requests.get(summary_url, params=params)
6         response.raise_for_status()
7         summary_data = response.json()
```

Ukázka kódu 6: Použití formátu JSON

V ukázce 7 je příklad odpovědi funkce `fetch_clinvar_variant_info()`.

```
1 {
2     "title": "NM_000021.4(PSEN1):c.779C>T (p.Ala260Val)",
3     "id": "98075",
4     "germline_classification": "Pathogenic",
5     "disease_associations": [
6         "Frontotemporal dementia",
7         "Acne inversa, familial, 3",
8         "Pick disease",
9         "Alzheimer disease 3"
10    ]
11 }
```

Ukázka kódu 7: Příklad JSON odpovědi

4.4 HTML, CSS

Když byla funkcionalita programu hotová v terminálu Pythonu, bylo třeba se zamyslet nad tím, jak ji co nejlépe zpřístupnit uživateli. Pro mě byla odpověď jasná – pokud si chci něco vyzkoušet, nechci si zbytečně nic instalovat. Proto jsem se rozhodla vytvořit webovou aplikaci, ke které se uživatel dostane jednoduše přes prohlížeč.

Pro vytvoření základní struktury uživatelského rozhraní aplikace jsem využila HTML5 (*HyperText Markup Language*) a CSS (*Cascading Style Sheets*), které se starají o její vzhled a stylování.

4.5 JavaScript

JavaScript je klíčovou součástí většiny moderních webových aplikací. Na rozdíl od Pythonu, který se běžně používá na serverové straně, JavaScript umožňuje

interaktivitu a dynamické chování přímo v prohlížeči, například aktualizaci obsahu stránky bez nutnosti jejího opětovného načítání.

Při návrhu aplikace jsem se rozhodla, že funkce na straně backendu (v Pythonu) budou co nejuniverzálnější, aby je bylo možné snadno využít v dalších projektech. Z tohoto důvodu je JavaScript na straně klienta poměrně robustní, aby efektivně komunikoval s backendem a zajišťoval plynulý chod aplikace.

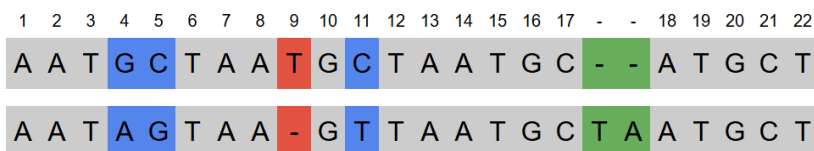
V aplikaci GenAnalyzer jsem JavaScript využila pro následující úkoly:

- **Dynamické prvky a interaktivita** - Pomocí event listenerů detekují akce uživatele, jako je kliknutí na tlačítko.
- **Manipulace s DOM⁵** - JavaScript umožňuje dynamickou aktualizaci obsahu webové stránky bez jejího opětovného načítání, například zobrazení nebo skrytí dat podle uživatelského vstupu.
- **Asynchronní komunikace (AJAX⁶)** - Pro komunikaci s backendem používám asynchronní ⁷ HTTP požadavky, které umožňují posílat data na server a získávat odpovědi bez nutnosti opětovného načítání stránky. [26]

4.5.1 D3.js

D3.js je JavaScriptová knihovna, která slouží k vizualizaci dat na webových stránkách. Na rozdíl od jiných knihoven, které mohou používat vlastní grafické formáty nebo technologie, D3.js využívá standardní webové technologie, jako jsou SVG (*Scalable Vector Graphics*), Canvas a HTML. Název D3 je zkratka pro data-driven documents, kde „dokumenty“ odkazují na standard Document Object Model (DOM), který reprezentuje obsah webových stránek.[28]

V aplikaci *Advanced Sequence Aligner* používám D3.js knihovnu k barevnému vykreslování rozdílů v zarovnaných sekvencích ve formátu SVG, což můžete vidět na obrázku 1.



Obrázek 1: Vykreslení zarovnaných sekvencí

SVG (*Scalable Vector Graphics*) je textově definovaný formát pro vektorovou grafiku, podporovaný webovými prohlížeči. V D3.js se SVG používá k vizualizaci dat na webových stránkách. Na rozdíl od vykreslování tvarů pomocí běžných HTML elementů, jako jsou `<div>` nebo `<span>`, které nejsou určeny pro práci s grafikou, je práce se SVG v D3.js efektivnější a vizuálně konzistentní.[29]

⁵DOM (*Document Object Model*) je programové rozhraní, které umožňuje programům měnit strukturu, styl a obsah HTML dokumentů.

⁶AJAX (*Asynchronous JavaScript and XML*) je technologie umožňující provádět HTTP požadavky přímo z JavaScriptu bez nutnosti znovunačtení stránky.

⁷Asynchronní komunikace umožňuje odesílání požadavků na server a přijímání odpovědí bez blokování hlavního vlákna aplikace. To znamená, že uživatel může pokračovat v interakci se stránkou, zatímco se data načítají na pozadí.

4.6 WSGI server

Flask je WSGI (*Web Server Gateway Interface*) framework, což znamená, že k jejímu spuštění je potřeba WSGI server. Tradiční webový server nerozumí Pythonu nebo nemá žádný způsob, jak spouštět Python aplikace.

Když server WSGI obdrží požadavek klienta, předá ho aplikaci a poté odešle odpověď zpět klientovi. Tento proces můžete vidět na schématu 2. [30]

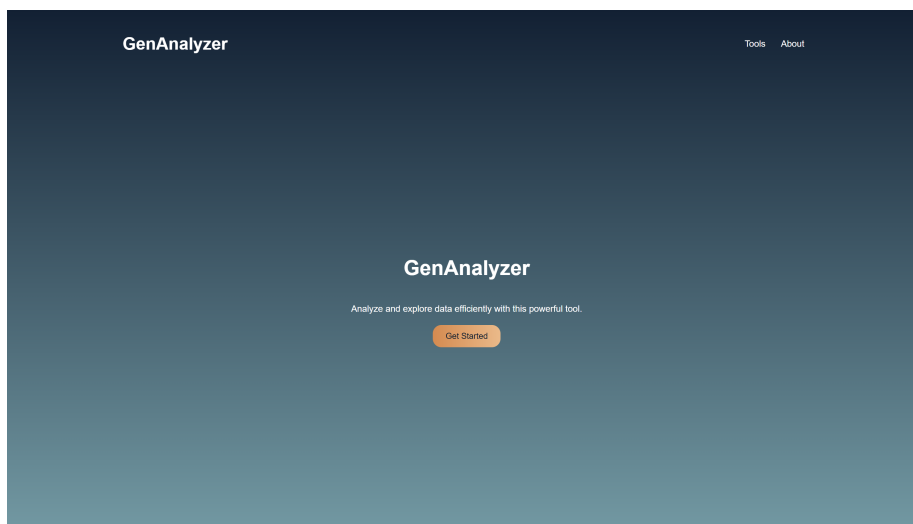


Obrázek 2: Schéma znázorňující proces komunikace

Existuje mnoho služeb, které umožňují hostování webových aplikací, aniž byste museli spravovat vlastní server, doménu atd., a které používají WSGI server nebo podobné rozhraní. Já pro webhosting používám *PythonAnywhere*, což je online integrované vývojové prostředí a služba pro webhosting, která poskytuje přístup k Pythonu a příkazovému řádku na serverech přímo v prohlížeči. [31]

5 Implementace

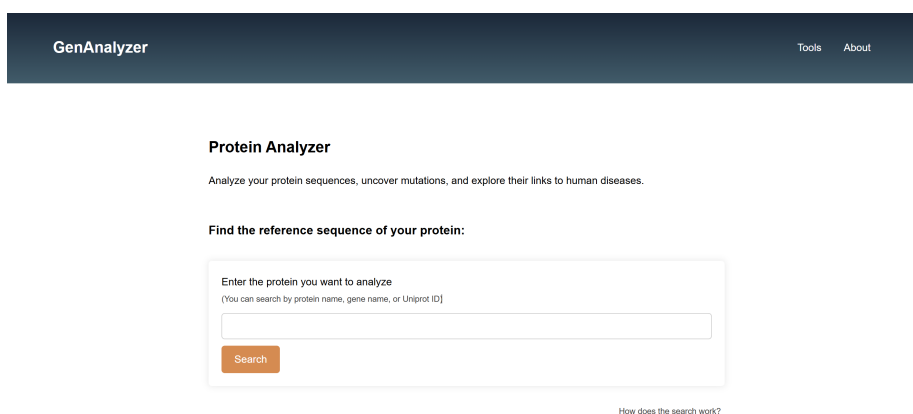
Aplikace je postavena na frameworku Flask a skládá se ze dvou hlavních částí: Protein Analyzer a Advanced Sequence Aligner. Tato kapitola popisuje klíčové funkcionality aplikace, včetně zpracování sekvencí, detekce mutací a komunikace s externími databázemi.



Obrázek 3: Úvodní stránka webové aplikace

5.1 Vyhledání v databázi UniProt

Vyhledávání proteinů v databázi UniProt probíhá prostřednictvím webového formuláře.



Obrázek 4: Sekce pro vyhledání referenčního proteinu

Po odeslání formuláře je asynchronně odeslán požadavek na server na endpoint `/search_gene` pomocí metody `fetch`⁸.

```
1 const response = await fetch(`/search_gene?gene=${geneName}`);
2 if (!response.ok) {
3     throw new Error(`HTTP error! Status: ${response.status}`);
4 }
5 const data = await response.json();
```

Ukázka kódu 8: Asynchronní získání výsledků vyhledávání

Python funkce `search_gene()` získává výsledky pomocí podfunkcí `get_search_results()` a `get_disease_involvement()`, které provádějí vyhledávání v UniProt REST API.

Funkce `get_search_results()` slouží k vyhledání proteinů v databázi UniProt na základě zadaného názvu genu. Vyhledává pouze recenzované (reviewed) lidské proteiny (Homo sapiens, `organism_id:9606`) a filtruje ty, které mají přiřazenou informaci o zapojení do nemocí.

Pro každý nalezený protein se ověřuje, zda je spojen s nemocemi, pomocí funkce `get_disease_involvement()`, která extrahuje odpovídající ID nemocí. Implementace této funkce je uvedena v ukázce kódu 9.

```
1 def get_disease_involvement(accession):
2     try:
3         response = requests.get(f"{UNIPROT_URL}/{accession}.json")
4         response.raise_for_status()
5         detailed_data = response.json()
6         disease_ids = []
7         for comment in detailed_data.get("comments", []):
8             if comment.get("commentType") == "DISEASE":
9                 disease_info = comment.get("disease", {})
10                 if "diseaseId" in disease_info:
11                     disease_ids.append(disease_info["diseaseId"])
12         return disease_ids
13     except requests.exceptions.RequestException as e:
14         return [f"Failed to fetch detailed data: {str(e)}"]
```

Ukázka kódu 9: Extrakce nemocí pro daný protein z UniProt

Server vrací výsledky jako seznam slovníků, které obsahují:

- `accession` – unikátní ID proteinu v UniProt,
- `protein_name` – název proteinu,
- `disease_involvement` – seznam nemocí spojených s proteinem.

JavaScript na straně klienta zpracovává seznam a generuje HTML prvky pro zobrazení informací o každém nalezeném proteinu. Kliknutím na název proteinu se zobrazí sekce pro analýzu sekvencí a automaticky se spustí funkce `fetchFastaSequence()`, která získá referenční sekvenci proteinu. Tato funkce provádí asynchronní požadavek na server a získá referenční sekvenci, kterou následně vloží do příslušného HTML prvku.

⁸Metoda `fetch` slouží k asynchronnímu načítání zdrojů ze sítě. Přijímá parametr určující požadovaný zdroj a vrací objekt představující odpověď na požadavek.^[32]

Funkce `get_protein_fasta()` (viz ukázka kódu 10) získá sekvenci proteinu přímo přes UniProt API ve FASTA formátu a odstraní hlavičku, typickou pro tento formát (první řádek začínající ">"). Výsledkem je čistá sekvence aminokyselin, která je použita pro další analýzu.

```

1 def get_protein_fasta(accession):
2     fasta_url = f"{UNIPROT_URL}/{accession}.fasta"
3     try:
4         response = requests.get(fasta_url)
5         response.raise_for_status()
6         fasta_data = response.text
7         reference_sequence = extract_the_sequence(fasta_data)
8         return reference_sequence
9     except requests.exceptions.RequestException as e:
10        return f"Error fetching FASTA data: {str(e)}", 500

```

Ukázka kódu 10: Načtení referenční sekvence proteinu

5.2 Zarovnání sekvencí

Uživatel může zadat proteinovou sekvenci, kterou chce porovnat s referenční sekvencí, ručně do textového pole formuláře nebo ji nahrát z textového souboru. Aplikace podporuje textové formáty s těmito příponami: `.txt`, `.fasta`, `.fas`, `.fa`, `.faa`.

Obrázek 5: Sekce pro porovnání sekvencí

Po odeslání sekvence se na server odesílá HTTP POST požadavek na endpoint `/align_sequence` s referenční a vstupní sekvencí ve formátu JSON.

Na straně serveru probíhá následující zpracování:

1. Odstranění hlavičky formátu FASTA (pokud je přítomna).
2. Ověření, zda sekvence obsahuje pouze platné aminokyseliny.
3. Globální zarovnání referenční a vstupní sekvence pomocí funkce `align_sequences()` (viz ukázka kódu 1).

4. Výpočet procentuální shody sekvencí pomocí funkce `match_score()` (viz ukázka kódu 11).

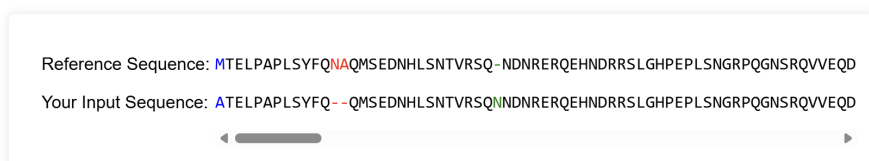
```
1 def match_score(alignment):
2     ref_aligned, seq_aligned = alignment[0], alignment[1]
3
4     matches = sum(1 for r, s in zip(ref_aligned, seq_aligned) if r == s)
5     total_length = len(ref_aligned)
6     match_percentage = round((matches / total_length) * 100, 2)
7
8     return match_percentage
```

Ukázka kódu 11: Výpočet procentuální shody sekvencí

Server následně vrátí odpověď ve formátu JSON obsahující:

- `aligned_reference_sequence` – zarovnanou referenční sekvenci,
- `aligned_input_sequence` – zarovnanou vstupní sekvenci,
- `match_percentage` – procentuální shodu sekvencí.

Pokud je shoda 100 %, aplikace uživateli oznámí, že sekvence jsou identické. Při nízké shodě (nižší než 50 %) se zobrazí varování s možností přesto zobrazit zarovnání. V ostatních případech se standardně zobrazí zarovnané sekvence, jak lze vidět na obrázku 6 .



Obrázek 6: Výpis zarovnaných sekvencí na stránce

5.3 Detekce mutací

Na základě zarovnaných sekvencí jsou detekovány mutace, tedy rozdíly ve vstupní a referenční sekvenci. Každý typ mutace (substituce, delece, insertce, extenze) je detekován samostatnou funkcí, což činí kód přehlednější a opětovně použitelný. Všechny čtyři funkce (`identify_substitution`, `identify_deletion`, `identify_insertion`, `identify_extension`) mají podobnou strukturu. Iterují přes zarovnané sekvence referenční a vstupní, porovnávají je a detekují konkrétní typ mutace.

5.3.1 Substituce

Funkce `identify_substitution()` porovnává zarovnané pozice v obou sekvencích a pokud dojde k nesouladu, kde žádná sekvence neobsahuje mezeru ("-"), je to považováno za substituci.

Zápis v HGVS formátu:

```
f"p.{get_three_letter_symbol(ref)}{ref_pos}  
{get_three_letter_symbol(input)}" 9
```

5.3.2 Delece

Detekce delecí je složitější, protože musí rozlišovat mezi jednotlivými a vícenásobnými delecemi. Funkce `identify_deletion()` detekuje delecí, když má pozice v referenční sekvenci aminokyselinu, ale odpovídající pozice ve vstupní sekvenci obsahuje mezeru ("-").

Zápis v HGVS formátu:

- jednobodová delece:
f"p.{get_three_letter_symbol(del_residue)}{del_start}del"
- vícenásobná delece:
f"p.{get_three_letter_symbol(deletion_seq[0])}{del_start}
_{get_three_letter_symbol(deletion_seq[-1])}{del_end}del"

5.3.3 Inzerce

Inzerce jsou detekovány, když referenční sekvence obsahuje mezeru ("-"), zatímco ve vstupní sekvenci je na této pozici aminokyselina. Funkce `identify_insertion()` detekuje jednotlivé i vícenásobné inzerce.

Zápis v HGVS formátu:

```
f"p.{get_three_letter_symbol(ref_seq[ref_pos-1])}{ref_pos}  
_{get_three_letter_symbol(ref_seq[ins_end-1])}  
{ins_end}ins{' '.join(three_letter_insertion_seq)}"
```

5.3.4 Extenze

Funkce `identify_extension()` detekuje extenze na N-terminálu (na začátku sekvence) nebo C-terminálu (na konci sekvence). Funkce se zaměřuje na to, zda se na začátku nebo na konci referenční sekvence nacházejí mezery, což naznačuje, že došlo k přidání aminokyselin.

Zápis v HGVS formátu:

- extenze na N-terminálu:
f"p.{get_three_letter_symbol(first_aa)}1ext-{i}"
- extenze na C-terminálu:
f"p.{get_three_letter_symbol(last_aa)}{last_aa_position}
{get_three_letter_symbol(new_aa_at_that_pos)}
ext{get_three_letter_symbol(new_last_aa)}{extension_length}"

⁹Funkce `get_three_letter_symbol()` převádí

5.3.5 Výstup a zobrazení výsledků

Každá funkce pro detekci mutací vrací seznam mutací, přičemž každá mutace je reprezentována slovníkem, který obsahuje následující údaje:

- **type** – typ mutace (např. "substitution"),
- **position** – pozice mutace v sekvenci (např. "275"),
- **change_one** – popis mutace s jednopísmennými zkratkami aminokyselin (např. "A > V"),
- **change_three** – popis mutace s třípísmennými¹⁰ zkratkami aminokyselin (např. "Ala > Val"),
- **hgvs** – zápis mutace ve standardizovaném formátu HGVS (např. "p.Ala275Val").

Výsledky detekce mutací se zobrazí na stránce ve formě tabulky, která přehledně shrnuje všechny nalezené mutace, jak je znázorněno na obrázku 7.

Filter by Type: all Sort by Position

Type	Position	Change (one-letter)	Change (three-letter)	HGVS
substitution	1	M > A	Met > Ala	p.Met1Ala
substitution	113	L > P	Leu > Pro	p.Leu113Pro
substitution	146	M > L	Met > Leu	p.Met146Leu
substitution	260	A > V	Ala > Val	p.Ala260Val
substitution	318	E > G	Glu > Gly	p.Glu318Gly
deletion	13 - 14	missing N - A	missing Asn - Ala	p.Asn13_Ala14del
insertion	between 29 - 30	added N	added Asn	p.Gln29_Asn30insAsn

Download variants table as comma-separated CSV, semicolon-separated CSV, TSV or Markdown Table

Obrázek 7: Zobrazení nalezených mutací na stránce

Tabulka obsahuje podrobnosti o všech identifikovaných mutacích ve vstupní sekvenci, včetně jejich zápisu ve formátu HGVS, který představuje standardní způsob dokumentace genetických změn.

5.4 Hledání asociací s nemocemi

Poslední z klíčových funkcionalit nástroje *Protein Analyzer* je vyhledávání asociací mezi jednotlivými mutacemi a známými genetickými chorobami. Tento

¹⁰Třípísmenné zkratky aminokyselin se používají pro zápis mutací, aby nedošlo k záměně s nukleotidy.

nástroj využívá data z klinické databáze ClinVar, která shromažďuje informace o genetických variantách a jejich vztahu k nemocem.

Po kliknutí na tlačítko pro zobrazení asociací s nemocemi se dynamicky generuje tabulka s výsledky. Pro každou variantu (mutaci) je na server odeslán požadavek, který obsahuje typ mutace, pozici mutace a její HGVS zápis. Server postupuje následujícím způsobem:

1. Získá ID proteinu uložené v aktuální session.
2. Pomocí funkce `fetch_clinvar_variant_id()` vyhledá odpovídající varianty v databázi ClinVar na základě ID proteinu a varianty.
3. Získá podrobnosti o každé variantě pomocí funkce `fetch_clinvar_variant_info()`.

Pokud daná varianta není v databázi ClinVar nalezena, server vrátí zprávu o neznámé variantě. Na základě získaných informací se sestaví odpověď obsahující detaily o variantě a souvisejících nemocech, která je následně odeslána zpět klientovi.

5.4.1 Komunikace s ClinVar API

Pro získání informací o genetických variantách a jejich vztahu s nemocemi používá server NCBI eUtils (*Entrez Programming Utilities*), konkrétně endpointy `search.fcgi` a `esummary.fcgi`.

Funkce `fetch_clinvar_variant_id()`, jejíž část můžete vidět v ukázce kódu 12, využívá endpoint `esearch.fcgi` pro vyhledávání varianty podle ID proteinu a HGVS zápisu varianty.

```
1 CLINVAR_URL = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/"
2
3 def fetch_clinvar_variant_id(protein_id, variant):
4     search_url = f"{CLINVAR_URL}esearch.fcgi"
5     params = {
6         "db": "clinvar",
7         "term": f"{protein_id}:{variant}",
8         "retmode": "json",
9         "api_key": API_KEY,
10    }
11    try:
12        search_response = requests.get(search_url, params=params)
13        search_response.raise_for_status()
14        search_data = search_response.json()
```

Ukázka kódu 12: Vyhledávání varianty v ClinVar

Parametry požadavku:

- `db`: Určuje, že hledáme v databázi ClinVar.
- `term`: Vyhledávací termín, který obsahuje ID proteinu a variantu.
- `retmode`: Určuje formát výsledků (v tomto případě JSON).
- `api_key`: Klíč pro přístup k API.

Funkce vrací seznam ID variant, které odpovídají danému vyhledávacímu termínu. Vzhledem k tomu, že jedna mutace na úrovni aminokyseliny může být způsobena více různými nukleotidovými mutacemi, může v databázi ClinVar existovat více variant pro jednu změnu na úrovni proteinů.

Pokud jsou ID varianty nalezena, server volá funkci `fetch_clinvar_variant_info()` pro získání podrobností o každé variantě na základě jejího ID pomocí endpointu `esummary.fcgi`.

Odpověď této funkce obsahuje:

- **Klasifikace varianty:** Popis klasifikace je získán z podpole "description", které se nachází v poli "germline_classification". Tato klasifikace určuje, zda je varianta považována za patologickou, benigní, nebo jinou. (viz ukázka kódu 13)
- **Asociace s nemocemi:** Informace o asociacích jsou uloženy v poli "trait_set", které obsahuje seznam nemocí spojených s danou variantou. Pokud má trait platný název, tento název je přidán do seznamu asociací. Filtruje se také název "not provided", protože to je hodnota, která se může objevit, pokud nemoc není uvedena. (viz ukázka kódu 14)

```
1 germline_info = variant_data.get("germline_classification", {})  
2 classification_description = germline_info.get("description", "N/A")
```

Ukázka kódu 13: Získání popisu klasifikace varianty

```
1 disease_associations = [  
2     trait.get("trait_name", "Unknown")  
3     for trait in germline_info.get("trait_set", [])  
4     if trait.get("trait_name") and  
5         trait["trait_name"].lower() != "not provided"  
6 ]  
7 disease_associations = list(set(disease_associations))
```

Ukázka kódu 14: Extrahování seznamu nemocí spojených s variantou

5.4.2 Zobrazení výsledků

Jakmile server získá a zpracuje všechny potřebné informace o jedné mutaci, vrátí je zpět do klientské aplikace, kde jsou postupně zobrazeny v tabulce, jak lze vidět na obrázku 8.

Pokud jsou nalezeny asociace, zobrazí se v tabulce název varianty, její ClinVar ID s odkazem na zápis varianty v ClinVar databázi, klasifikace závažnosti varianty a asociace s nemocemi. Odkaz umožňuje uživatelům snadno přistupovat k dalším informacím o variantě.

5.5 Advanced Sequence Aligner

Kromě hlavního nástroje *Protein Analyzer* obsahuje aplikace také nástroj *Advanced Sequence Aligner*. Ten umožňuje porovnávání libovolných dvou sekvencí (aminokyselin i nukleotidů) s možností přizpůsobení parametrů.

Filter by Variant Status: known Show/Hide Columns

Variant identifier	ClinVar ID	Significance	Disease Associations
NM_000021.4(PSEN1):c.338T>C (p.Leu113..	18145	Likely pathogenic	Alzheimer disease 3 Frontotemporal dementia Acne inversa, familial, 3 Pick disease
NM_000021.4(PSEN1):c.436A>T (p.Met146..	98828	Pathogenic	Pick disease
NM_000021.4(PSEN1):c.436A>C (p.Met146..	18123	Pathogenic	Alzheimer disease 3 Frontotemporal dementia Acne inversa, familial, 3 Pick disease
NM_000021.4(PSEN1):c.779C>T (p.Ala260..	98875	Pathogenic	Alzheimer disease 3 Frontotemporal dementia Acne inversa, familial, 3 Pick disease
NM_000021.4(PSEN1):c.953A>G (p.Glu318..	98894	Benign/Likely benign	Dilated cardiomyopathy 1U not specified Alzheimer disease 3 Alzheimer disease Pick disease Frontotemporal dementia Acne inversa, familial, 3

Download disease associations table as comma-separated CSV, semicolon-separated CSV, TSV or Markdown Table

Obrázek 8: Zobrazení asociací mutací s nemocemi

5.5.1 Přizpůsobení parametrů zarovnání

Advanced Sequence Aligner umožňuje uživatelům upravit různé parametry zarovnání, jako jsou hodnoty skóre nebo penalizace za mezery, což je nezbytné pro dosažení co nejpřesnějších výsledků v závislosti na typu sekvencí (DNA, RNA nebo proteinové sekvence).

Mezi parametry, které lze přizpůsobit, patří:

- **mode** – mód zarovnání (global, local)¹¹,
- **match_score** – skóre pro shodné nukleotidy nebo aminokyseliny,
- **mismatch_score** – penalizace za rozdílné nukleotidy nebo aminokyseliny,
- **open_gap_score** – penalizace za otevření mezery v zarovnání,
- **extend_gap_score** – penalizace za prodloužení již existující mezery.

Po provedení zarovnání se výsledky zobrazí ve formátu, kde lze snadno identifikovat mutace v porovnávaných sekvencích. Uživatelé tak mohou snadno pozorovat, jak změny parametrů ovlivňují výsledky zarovnání, což umožňuje lépe pochopit principy a algoritmy, které stojí za porovnáváním sekvencí a umožňují tento nástroj využít také pro výukové účely v oblasti bioinformatiky.

5.5.2 Získání dat z nástroje Protein Analyzer

Do nástroje *Advanced Sequence Aligner* lze vstoupit přímo z *Protein Analyzeru* pomocí odkazu po provedení zarovnání sekvencí. Uživatel si tak může dodatečně upravit parametry zarovnání přímo pro tyto dvě sekvence, které již analyzoval.

¹¹**global** – zarovná celou délku obou sekvencí, **local** – hledá nejlepší možnou shodu mezi částmi sekvencí[18]

Advanced Sequence Aligner

This tool allows you to input two sequences and align them using the `Bio.Align.PairwiseAligner` class from the Biopython library, which is also used in the Protein Analyzer tool of the GenAnalyzer web app.

Modify alignment parameters and observe how the results change, helping you understand the principles behind sequence comparison. You have complete control over the alignment process, with no need for programming knowledge.

Whether you're working with DNA, RNA, or protein sequences, this tool helps you analyze similarities and differences with precision.

Enter two sequences you want to align:
(Enter either a nucleotide or an amino acid sequence.)

Paste your first sequence here in FASTA format or as a raw sequence...

Paste your second sequence here in FASTA format or as a raw sequence...

Change the alignment parameters:
(These parameters are set to match those used in the Protein Analyzer tool. Feel free to adjust them as needed to explore different alignment results.)

mode = global ▼ ⓘ

Obrázek 9: Sekce *Advanced Sequence Aligner*

Po kliknutí na odkaz proběhnou následující kroky:

- Získání referenční sekvence proteinu a sekvence, kterou uživatel zadal do textového pole.
- Uložení sekvencí do `sessionStorage`, což znamená, že data budou dostupná v rámci aktuální relace prohlížeče¹². (viz ukázka kódu 15)
- Otevření nové stránky pro zarovnání s URL `/aligner` v nové záložce.

```

1 document.getElementById("link").addEventListener("click", function () {
2     const referenceSeq =
3         document.querySelector("#referenceSequence pre").textContent;
4     const userSeq = document.getElementById("sequenceInput").value;
5     sessionStorage.setItem("referenceSequence", referenceSeq);
6     sessionStorage.setItem("userSequence", userSeq);
7     window.open("/aligner", "_blank");
8 });

```

Ukázka kódu 15: Uložení sekvencí do `sessionStorage`

Na stránce `/aligner` se pak načtou sekvence z předchozí stránky, které byly uloženy do `sessionStorage`, a jsou vloženy do textových polí. Po načtení sekvencí se položky ze `sessionStorage` vymažou, aby nedocházelo k nechtěnému uchovávání těchto dat. (viz ukázka kódu 16)

```

1 document.addEventListener("DOMContentLoaded", function () {
2     // Retrieve sequences from localStorage
3     const referenceSeq =
4         sessionStorage.getItem("referenceSequence") || "";

```

¹²**Relace** je dočasné spojení mezi klientem a serverem. Data uložená do `sessionStorage` zůstanou dostupná pouze po dobu této relace, tedy dokud uživatel nezavře okno nebo kartu prohlížeče.[27]

```

5      const userSeq = sessionStorage.getItem("userSequence") || "";
6
7      // Populate textareas
8      document.getElementById("inputSequence1").value = referenceSeq;
9      document.getElementById("inputSequence2").value = userSeq;
10
11     // Clear localStorage after retrieval to prevent persisting data
12     sessionStorage.removeItem("referenceSequence");
13     sessionStorage.removeItem("userSequence");
14 });

```

Ukázka kódu 16: Načtení sekvencí a vymazání sessionStorage

Tento přístup umožňuje přechod mezi nástroji a poskytuje uživatelům vyšší kontrolu nad zarovnáním sekvencí.

6 Výstup a vizualizace

V této kapitole se zaměřím na výstupy a vizualizaci genetických dat v aplikaci, především na vizualizaci zarovnání sekvencí a zobrazení tabulek s mutacemi a asociacemi s chorobami. Tyto vizualizace jsou klíčové pro analýzu a interpretaci genomických dat, protože umožňují snadno identifikovat rozdíly mezi sekvencemi a poskytují přehledné zobrazení mutací a jejich charakteristik.

6.1 Vizualizace zarovnání sekvencí

Vizualizace zarovnání dvou sekvencí je důležitým nástrojem pro pochopení rozdílů mezi těmito sekvencemi. V aplikaci jsou specifické změny mezi dvěma sekvencemi vyznačeny vždy konkrétní barvou:

- Delece (červeně) - odstranění nukleotidu nebo aminokyseliny.
- Inzerce (zeleně) - vložení nukleotidu nebo aminokyseliny.
- Substituce (modře) - náhrada nukleotidu nebo aminokyseliny za jiný.

6.1.1 Funkce `drawAlignment()`

V nástroji Protein Analyzer vykresluje zarovnání sekvencí funkce `drawAlignment()`. Tato funkce zpracovává zarovnané sekvence znak po znaku a vykresluje je s barevným zvýrazněním:

- Pokud je znak ve vstupní sekvenci "-" (delece), odpovídající znak v referenční sekvenci je zobrazen červeně.
- Pokud je znak v referenční sekvenci "-" (inzerce), odpovídající znak ve vstupní sekvenci je zobrazen zeleně.
- Pokud se znaky v obou sekvencích liší (ale nejsou to mezery), označí se modře (substituce).
- Pokud jsou znaky shodné, zůstanou bez jakéhokoliv barevného zvýraznění.

```
1 if (inputChar === "-") {
2     // Deletion
3     formattedAlignedReferenceSequence +=
4         `<span style="color: red;">${refChar}</span>`;
5     formattedAlignedInputSequence +=
6         `<span style="color: red;">${inputChar}</span>`;
7 } else if (refChar === "-") {
8     // Insertion
9     formattedAlignedReferenceSequence +=
10        `<span style="color: green;">${refChar}</span>`;
11    formattedAlignedInputSequence +=
12        `<span style="color: green;">${inputChar}</span>`;
13 } else if (refChar !== inputChar) {
14     // Substitution
15    formattedAlignedReferenceSequence +=
16        `<span style="color: blue;">${refChar}</span>`;
```

```

17     formattedAlignedInputSequence +=
18         `<span style="color: blue;">${inputChar}</span>`;
19 } else {
20     formattedAlignedReferenceSequence += refChar;
21     formattedAlignedInputSequence += inputChar;
22 }

```

Ukázka kódu 17: Formátování sekvenčních změn

Formátované sekvence, které obsahují HTML značky pro barevné zvýraznění, jsou zobrazeny v elementech `<pre>`, které uchovávají formátování textu, čímž se dosahuje přehledného a snadno čitelného zobrazení sekvencí. Výstup formátování můžete vidět na obrázku 6.

6.1.2 Funkce `drawD3Alignment()`

V nástroji *Advanced Sequence Aligner* je vizualizace zarovnání realizována pomocí knihovny D3.js, která generuje SVG (*Scalable Vector Graphics*) pro zobrazení změn mezi sekvencemi. Každá změna (delece, inserce, substituce) je znázorněna pomocí barevně odlišených obdélníků a písmen.

Vykreslování SVG je založeno na XML (*Extensible Markup Language*), což znamená, že každý prvek v SVG je definován pomocí značkovacího kódu a atributů, podobně jako v HTML. Místo toho, aby byl obrázek vykreslován jako soubor s rastrovými daty (např. JPEG nebo PNG), SVG obsahuje textově definovaný obsah, který popisuje, jaký prvek a jakým způsobem by měl být zobrazen. [29]

D3.js umožňuje vytvářet SVG prvky přímo v kódu a upravovat je na základě dat. Tento proces zahrnuje dva hlavní kroky:

1. **Vytváření elementů:** Pomocí metody `.append()` jsou přidávány nové SVG prvky (např. `<rect>` nebo `<text>`).
2. **Nastavování atributů:** Pomocí metody `.attr()`, se definují vlastnosti prvků, jako jsou barva, velikost nebo umístění na obrazovce.

Ve funkci `drawD3Alignment()` jsou nejprve sekvence převedeny na pole objektů, přičemž každý objekt představuje datový bod¹³. Ten obsahuje informace o pozici, znaku referenční sekvence, znaku vstupní sekvence a typu shody mezi těmito znaky, podobně jako ve funkci `drawAlignment()`.

Obdélníky jsou vykresleny pro každý znak v referenční a vstupní sekvenci. Barva obdélníku odpovídá typu změny (shoda, delece, substituce, inserce).

```

1 svg.selectAll("rect.ref")
2   .data(data)
3   .enter()
4   .append("rect")
5   .attr("x", (d, i) => i * charWidth)
6   .attr("y", 10 + verticalPadding)
7   .attr("width", charWidth)
8   .attr("height", charHeight)

```

¹³Datový bod je objekt, který obsahuje informace o konkrétním znaku v sekvenci a o jeho porovnání s referenční sekvencí. Tato data jsou použita k určení, jaký grafický prvek (například obdélník) má být vykreslen pro daný znak.

```
9 | .attr("fill", d => colorScale[d.match]);
```

Ukázka kódu 18: Vykreslení obdélníku pomocí D3.js

V tomto kódu:

- `.data(data)` propojuje data (datové body) s grafickými prvky,
- `.append("rect")` přidává nový obdélník `<rect>` do SVG kontejneru,
- `.attr("x", (d, i) => i * charWidth)` nastavuje pozici na ose X na základě indexu datového bodu a šířky znaku,
- `.attr("fill", d => colorScale[d.match])` určuje barvu obdélníku na základě typu změny, `colorScale` je mapa přiřazující barvy pro jednotlivé typy změn.

Výstup vykreslení můžete vidět na obrázku 1.

6.2 Zobrazení tabulek mutací a asociací

Výsledky vyhledávání mutací a jejich asociací s nemocemi se uživateli zobrazují ve formě tabulek. Ty umožňují efektivní filtrování, řazení a stahování dat pro další analýzu či výzkum.

6.2.1 Filtrace podle typu mutace

Funkce `addTypeFilter()` umožňuje filtrovat mutace na základě jejich typu (delece, inserce, substituce) prostřednictvím rozbalovací nabídky. Uživatel si může vybrat požadovaný typ, nebo zvolit „all“ pro zobrazení všech variant. Tento filtr usnadňuje analýzu specifických změn v genetických sekvencích.

6.2.2 Řazení tabulky podle pozice mutace

Funkce `addSortButton()` přidává tlačítko pro řazení mutací podle jejich pozice v sekvenci. Používá pomocnou funkci `parsePosition()`, která správně interpreтуje číselné hodnoty pozic mutací, čímž zajišťuje správné seřazení tabulky.

6.2.3 Filtrace podle stavu varianty

Funkce `addDiseaseTableKnownFilter()` přidává rozbalovací nabídku pro filtrování variant podle jejich stavu.¹⁴ Možnosti zahrnují:

- all (všechny varianty),
- known (známé varianty),
- not known (neznámé varianty).

Při změně filtru se spustí funkce `filterKnownVariants()` (viz ukázka kódu 19), která na základě vybraného stavu skryje nebo zobrazí odpovídající řádky tabulky.

¹⁴Stav varianty je určen hodnotou ve sloupci "Variant identifier". Pokud tento sloupec obsahuje text "Not known variant", je považována za neznámou variant, v jiném případě je považována za variantu známou.

```

1 rows.forEach((row, index) => {
2   if (index === 0) return;
3   const variantIdentifier = row.cells[3].textContent;
4   const isKnownVariant =
5     !variantIdentifier.includes("Not known variant");
6
7   if (selectedStatus === "all" ||
8       (selectedStatus === "known" && isKnownVariant) ||
9       (selectedStatus === "not known" && !isKnownVariant)) {
10     row.style.display = '';
11   } else {
12     row.style.display = 'none';
13   }
14 }

```

Ukázka kódu 19: Filtrace odpovídajících řádků

6.2.4 Rozšířené filtrování sloupců

Funkce `addDiseaseTableColumnFilter()` umožňuje uživatelům zobrazit či skrýt konkrétní sloupce v tabulce pomocí checkboxů. Funkce vytváří tlačítko pro zobrazení/skrutí sloupců, které po kliknutí zobrazí checkboxy pro každý sloupec tabulky. Checkboxy umožňují uživateli vybrat, které sloupce budou zobrazeny. Při změně stavu checkboxu se volá funkce `toggleColumnVisibility()` (viz ukázka 20).

```

1 function toggleColumnVisibility(event) {
2   const columnIndex = event.target.dataset.columnIndex;
3   const table = document.getElementById('diseaseAssociationTable');
4   if (!table) return;
5   table.querySelectorAll('tr').forEach(row => {
6     const cell = row.cells[columnIndex];
7     if (cell) {
8       cell.style.display = event.target.checked ? "" : "none";
9     }
10  });
11 }

```

Ukázka kódu 20: Zobrazení nebo skrytí sloupce na základě výběru

Funkce `toggleColumnVisibility()` slouží k zobrazení nebo skrytí konkrétního sloupce v tabulce na základě uživatelského výběru. Funkce zjistí, jaký sloupec má být ovlivněn (na základě indexu sloupce z checkboxu), a upraví viditelnost všech buněk v daném sloupci (v každém řádku tabulky).

Tento mechanismus umožňuje uživatelům přizpůsobit tabulku podle vlastních potřeb bez nutnosti jejího opětovného načítání, což je užitečné zejména u velkých tabulek s mnoha datovými sloupci.

6.2.5 Export dat

Pro další analýzu a sdílení výsledků aplikace umožňuje export tabulek do několika formátů:

- **CSV** (oddělené čárkami nebo středníky),
- **TSV** (oddělené tabulátory),
- **Markdown, MD** (s " | " oddělovači).

To poskytuje uživatelům flexibilitu při práci s daty podle jejich potřeby a umožňuje snadno přenést data do dalších nástrojů pro analýzu.

Po kliknutí na tlačítko stažení se spustí funkce `downloadDiseaseTable()`, která exportuje tabulku do zvoleného formátu. Funkce přijímá následující parametry:

- **table** – HTML element tabulky k exportu,
- **delimiter** – oddělovač hodnot (např. čárka pro CSV, tabulátor pro TSV),
- **extension** – požadovaný formát souboru (`csv`, `tsv`, `md`).

Funkce projde všechny řádky a buňky tabulky (viz ukázka **21**). Problém nastává, pokud buňka HTML tag `<br>`. Tento tag jsem se pro zajištění správného formátování rozhodla nahradit středníkem. Pokud jde o CSV, text je navíc uzavřen do uvozovek pro správný zápis.

```

1 rows.forEach((row, index) => {
2   let rowData = [];
3   row.querySelectorAll("th, td").forEach(cell => {
4     let cellText = cell.textContent.trim();
5     if (cell.innerHTML.includes("<br>")) {
6       cellText = cell.innerHTML.replace(/<br\s*\/*>/g, " ");
7     }
8     rowData.push(cellText);
9   });
10  content += `| ${rowData.join(delimiter)} |\n`;
11 });

```

Ukázka kódu 21: Procházení řádků a buněk tabulky

Vygenerovaný obsah je zapsán do objektu typu `Blob` (*Binary Large Object*), což jsou binární data, která lze stáhnout jako soubor. Funkce automaticky vytvoří odkaz pro stažení a spustí stahování souboru s příponou podle požadovaného formátu (`.csv`, `.tsv` nebo `.md`). [33]

```

1 const type = extension === "tsv"
2   ? "text/tab-separated-values" : extension === "md"
3   ? "text/markdown" : "text/csv";
4 const blob = new Blob([content], { type: `${type};charset=utf-8;` });
5 const link = document.createElement("a");
6 link.href = URL.createObjectURL(blob);
7 link.download = `diseaseassociationstable.${extension}`;
8 link.click();

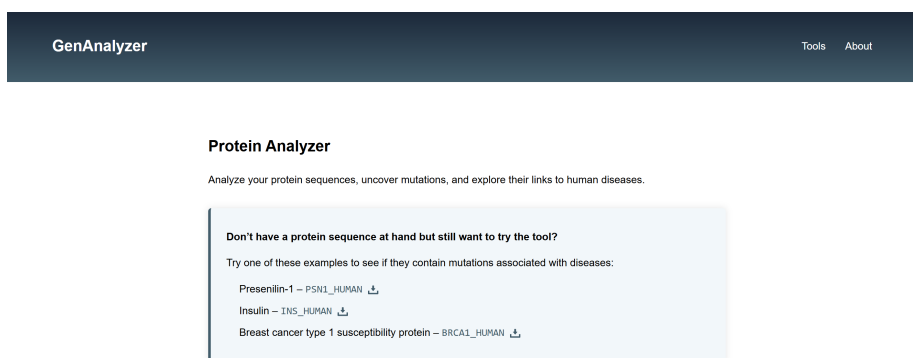
```

Ukázka kódu 22: Vytvoření `Blob` objektu a stažení souboru

7 Zpětná vazba

Po zveřejnění aplikace jsem oslovila několik odborníků i studentů z oblasti bioinformatiky s cílem získat zpětnou vazbu, která by přispěla dalšímu zlepšení funkčnosti a použitelnosti nástroje. Reakce uživatelů byly velmi přínosné a vedly k několika konkrétním úpravám aplikace.

Jednou z nejčastějších připomínek byla absence testovacích vstupů, které by umožnily uživatelům snadno si aplikaci vyzkoušet. Na základě tohoto podnětu jsem do aplikace přidala ukázkové datové soubory ke stažení. Tyto soubory nyní slouží nejen k testování, ale zároveň i k edukačním účelům – uživatelé se díky nim mohou seznámit s konkrétními proteiny a jejich souvislostmi s vybranými onemocněními.

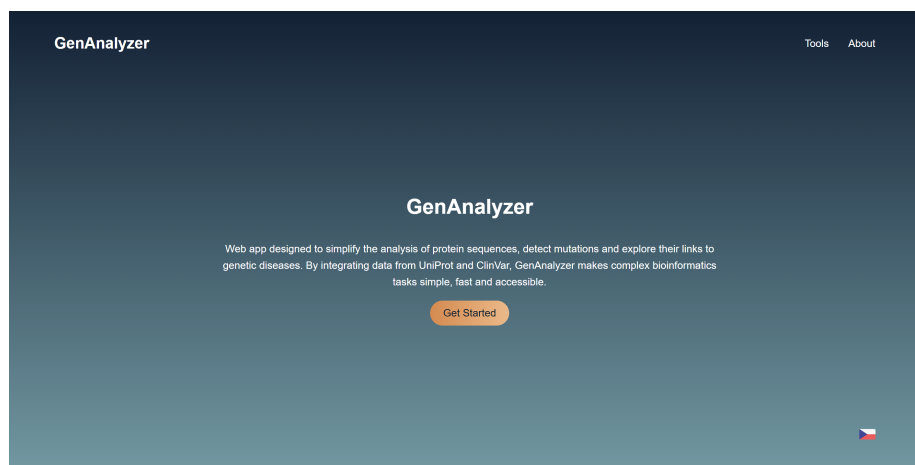


Obrázek 10: Přidané testovací soubory ke stažení

Významnou zpětnou vazbu mi poskytl Mgr. David Procházka, výzkumný specialista, který se mimo jiné podílel na vývoji webového vyhledávače strukturních podobností proteinů AlphaFind. Na základě jeho doporučení jsem upravila úvodní stránku aplikace a doplnila ji o detailnější popis jejího fungování. Zároveň jsem provedla úpravy v oblasti zobrazení výsledků. Některým uživatelům vadila nutnost ročního rozklikávání jednotlivých částí výsledků. Vzhledem k tomu, že analýza probíhá velmi rychle a počet výstupů (například nalezených mutací) bývá obvykle nízký, rozhodla jsem se výsledky zobrazovat automaticky v plném rozsahu bez nutnosti další interakce ze strany uživatele.

Velmi cenné bylo rovněž doporučení od doc. RNDr. Barbory Kozlíkové, Ph.D., proděkanky pro výzkum a doktorské studium FI MU a vedoucí Laboratoře vizualizace. Navrhla implementaci interaktivního propojení zarovnaných sekvencí s tabulkovými výstupy, což by uživatelům usnadnilo orientaci v analyzovaných datech a napomohlo jejich lepšímu porozumění. Tuto funkcionalitu plánuji zařadit v budoucích verzích aplikace.

Měla jsem také možnost aplikaci osobně probrat s Mgr. Monikou Čechovou, Ph.D., odbornou asistentkou na FI MU, která se věnuje propojení genomiky s moderními výpočetními metodami. Ta ocenila především potenciál nástroje jako výukové pomůcky pro studenty připravovaného bakalářského programu bioinformatika – zejména při porozumění standardizovanému HGVS formátu při popisu mutací.



Obrázek 11: Úvodní stránka s rozšířeným popiskem a tlačítkem pro překlad

Tato zpětná vazba potvrdila, že aplikace má potenciál nejen jako analytický nástroj, ale i jako prostředek pro edukaci. Na základě těchto podnětů jsem přidala podrobnější popisky tabulek a možnost přepnutí rozhraní do češtiny, neboť většina současných uživatelů pochází z českého prostředí. Díky těmto úpravám je aplikace nyní přístupnější i pro méně zkušené uživatele, kteří se s danou problematikou teprve seznamují.

Zpětná vazba se však netýkala pouze návrhů na zlepšení. Uživatelé ocenili zejména:

- přehlednost uživatelského rozhraní,
- úvodní instruktážní video,
- jednoduchost použití,
- integraci více datových zdrojů.

8 Závěr

Cílem této práce bylo vytvořit webovou aplikaci, která usnadňuje analýzu proteinových sekvencí, identifikaci mutací a jejich možných souvislostí s genetickými chorobami. Tento cíl byl úspěšně naplněn – vznikl nástroj, který zjednodušuje analýzu proteinů a díky propojení s klíčovými databázemi a intuitivnímu uživatelskému rozhraní aplikace zpřístupňuje komplexní procesy analýzy širšímu publiku a poskytuje rychlé, přesné výsledky.

Celá práce je navíc dostupná jako otevřený projekt na stránce <https://github.com/jajik123/GenAnalyzer>, což umožňuje její další využití, úpravy a rozšíření.

8.1 Budoucí rozvoj

Ačkoli aplikace splnila svůj primární účel, práce na tomto projektu stále pokračuje. V současnosti se soustředím na získávání zpětné vazby, která by mohla přispět k dalšímu vylepšení a rozšíření jejích funkcionalit. Níže uvádím několik možností pro její další vylepšení a rozšíření:

8.1.1 Rozšíření podpory dalších databází

Integrace s dalšími bioinformatickými databázemi, jako je Ensembl¹⁵ nebo COSMIC¹⁶, by rozšířila možnosti analýzy a validace mutací, čímž by aplikace poskytovala ještě ucelenější výsledky.

8.1.2 Podpora dalších typů sekvencí

V současné podobě se aplikace zaměřuje na analýzu proteinových sekvencí. Rozšíření podpory na DNA a RNA by umožnilo komplexnější analýzu mutací na úrovni nukleotidů a rozšířilo tak spektrum uživatelů.

8.1.3 Přihlášení uživatele

Dalším vylepšením by byla možnost přihlášení uživatele, který by si mohl ukládat své výsledky přímo v aplikaci. To by bylo výhodné pro uživatele, kteří by chtěli porovnávat různé výsledky analýzy sekvencí. Tento aspekt však částečně řeší již možnost stahování výsledků, což považuji za velký přínos, zejména ve srovnání s jinými nástroji tohoto typu, které stahování neumožňují.

¹⁵Ensembl je databáze genomových informací.

¹⁶The Catalogue Of Somatic Mutations In Cancer

9 Bibliografie

- [1] CVRČKOVÁ, Fatima. *Úvod do praktické bioinformatiky*. Praha: Academia, ©2006. ISBN 80-200-1360-1. [cit. 26. 3. 2025].
- [2] *Nukleová báze* [online]. Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, ©2025. Dostupné z: https://cs.wikipedia.org/wiki/Nukleov%C3%A1_b%C3%A1ze. [cit. 2025-03-26].
- [3] *GenScript Codon Table Tool* [online]. GenScript, ©2002-2025. Dostupné z: <https://www.genscript.com/tools/codon-table>. [cit. 2025-03-26].
- [4] *Genetický kód* [online]. WikiSkripta, ©2021. Dostupné z: https://www.wikiskripta.eu/w/Genetick%C3%BD_k%C3%B3d. [cit. 2025-03-26].
- [5] *Translace* [online]. WikiSkripta, ©2024. Dostupné z: <https://www.wikiskripta.eu/w/Translace>. [cit. 2025-03-26].
- [6] INTERNATIONAL UNION OF PURE AND APPLIED CHEMISTRY, INTERNATIONAL UNION OF BIOCHEMISTRY. *Nomenclature and symbolism for amino acids and peptides (Recommendations 1983)*. Pure and Applied Chemistry, 1984, roč. 56, č. 5, s. 595–624. Dostupné z: <https://publications.iupac.org/pac/1984/pdf/5605x0595.pdf>. [cit. 2025-03-26].
- [7] *Genové mutace* [online]. Mendelova univerzita v Brně, ©2020. Dostupné z: https://user.mendelu.cz/urban/vsg1/molekul/mol_mut.html. [cit. 2025-03-26].
- [8] ŠÍPEK, Antonín. *Mutace* [online]. Genetika - Biologie, ©2010 - 2014. Dostupné z: <http://www.genetika-biologie.cz/mutace>. [cit. 2025-03-26].
- [9] *Mutace* [online]. WikiSkripta, ©2025. Dostupné z: <https://www.wikiskripta.eu/w/Mutace>. [cit. 2025-03-26].
- [10] The UniProt Consortium. *UniProt: the Universal Protein Knowledgebase in 2025*. Nucleic Acids Research, 2025, roč. 53, s. D609–D617. Dostupné z: <https://doi.org/10.1093/nar/gkae1010>. [cit. 2025-03-26].
- [11] *Proteins API* [online]. EMBL-EBI, ©2025. Dostupné z: <https://www.ebi.ac.uk/proteins/api/doc/>. [cit. 2025-03-26].
- [12] *ClinVar* [online]. The National Center for Biotechnology Information (NCBI), ©2025. Dostupné z: <https://www.ncbi.nlm.nih.gov/clinvar/>. [cit. 2025-03-26].
- [13] *Entrez Programming Utilities Help* [online]. Bethesda (MD): National Center for Biotechnology Information (US), ©2010. Dostupné z: <https://www.ncbi.nlm.nih.gov/books/NBK25501/>. [cit. 2025-03-26].
- [14] *New API Keys for the E-utilities* [online]. NCBI Insights. The National Center for Biotechnology Information (NCBI), ©2017. Dostupné z: <https://ncbiinsights.ncbi.nlm.nih.gov/2017/11/02/new-api-keys-for-the-e-utilities/>

- [15] *FASTA format* [online]. Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, ©2024. Dostupné z: https://en.wikipedia.org/wiki/FASTA_format. [cit. 2025-03-26].
- [16] *FASTA Format for Nucleotide Sequences* [online]. The National Center for Biotechnology Information (NCBI), ©2021. Dostupné z: <https://www.ncbi.nlm.nih.gov/genbank/fastaformat/>. [cit. 2025-03-26].
- [17] DEN DUNNEN, Johan T., DALGLEISH, Raymond, MAGLOTT, Donna R., HART, Reid K., GREENBLATT, Marc S., MCGOWAN-JORDAN, Jean, ROUX, Anne-Françoise, SMITH, Terry, ANTONARAKIS, Stylianos E., TASCHNER, Peggy E. M. *HGVS Recommendations for the Description of Sequence Variants: 2016 Update*. Human Mutation, 2016, roč. 37, č. 6, s. 564–569. DOI: <https://doi.org/10.1002/humu.22981>. [cit. 2025-03-26].
- [18] *Biopython* [online]. ©2025. Dostupné z: <https://biopython.org/>. [cit. 2025-03-26].
- [19] *Flask Tutorial in Visual Studio Code* [online]. Visual Studio Code, ©2025. Dostupné z: <https://code.visualstudio.com/docs/python/tutorial-flask>. [cit. 2025-03-26].
- [20] *Flask* [online]. Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, ©2024. Dostupné z: <https://cs.wikipedia.org/wiki/Flask>. [cit. 2025-03-26].
- [21] *Nauč se Python* [online]. Dekorátory, ©2016-2017. Dostupné z: <https://naucese.python.cz/2019/brno-jaro-knihovny/beginners/decorators/>. [cit. 2025-03-26].
- [22] *Nauč se Python* [online]. Webové aplikace: Flask, ©2016-2017. Dostupné z: <https://naucese.python.cz/lessons/intro/flask/>. [cit. 2025-03-26].
- [23] *Python Flask-Flask Session* [online]. Spark Databox, ©2019. Dostupné z: <https://sparkdatabox.com/tutorials/python-flask/flask-session>. [cit. 2025-03-26].
- [24] Ecma International. *ECMA-404: The JSON Data Interchange Syntax*. 2nd ed. Geneva, Switzerland: Ecma International, ©2017. Dostupné z: <https://ecma-international.org/publications-and-standards/standards/ecma-404/>. [cit. 2025-03-26].
- [25] *JavaScript* [online]. Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, ©2025. Dostupné z: <https://cs.wikipedia.org/wiki/JavaScript>. [cit. 2025-03-26].
- [26] *AJAX v JavaScriptu* [online]. ITnetwork, ©2025. Dostupné z: <https://www.itnetwork.cz/javascript/oop/ajax-v-javascriptu-zakladni-dotazy>. [cit. 2025-03-26].
- [27] *Web storage* [online]. Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, ©2023. Dostupné z: https://cs.wikipedia.org/wiki/Web_storage. [cit. 2025-03-26].

- [28] Bostock, Mike *What is D3?* [online]. ©2025. Dostupné z: <https://d3js.org/what-is-d3>. [cit. 2025-03-26].
- [29] MURRAY, Scott. *Interactive data visualization for the web*. Sebastopol: O'Reilly, ©2013. ISBN 978-1-449-33973-9.
- [30] *WSGI Servers* [online]. Full Stack Python, ©2012–2022. Dostupné z: <https://www.fullstackpython.com/wsgi-servers.html>. [cit. 2025-03-26].
- [31] *PythonAnywhere* [online]. Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, ©2024. Dostupné z: <https://en.wikipedia.org/wiki/PythonAnywhere>. [cit. 2025-03-26].
- [32] *Window: fetch() method* [online]. MDN Web Docs, ©1998–2025. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/Window/fetch>. [cit. 2025-03-26].
- [33] *Blob* [online]. MDN Web Docs, ©1998–2025. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/Blob>. [cit. 2025-03-26].

Seznam obrázků

1	Vykreslení zarovnaných sekvencí	18
2	Schéma znázorňující proces komunikace	19
3	Úvodní stránka webové aplikace	20
4	Sekce pro vyhledání referenčního proteinu	20
5	Sekce pro porovnání sekvencí	22
6	Výpis zarovnaných sekvencí na stránce	23
7	Zobrazení nalezených mutací na stránce	25
8	Zobrazení asociací mutací s nemocemi	28
9	Sekce <i>Advanced Sequence Aligner</i>	29
10	Přidané testovací soubory ke stažení	36
11	Úvodní stránka s rozšířeným popiskem a tlačítkem pro překlad	37

Seznam ukázek kódu

1	Zarovnání sekvencí	14
2	Základní routing ve Flasku	15
3	Registrace Blueprintu	16
4	Použití Blueprintu pro organizaci jednotlivých cest aplikace	16
5	Použití session	16
6	Použití formátu JSON	17
7	Příklad JSON odpovědi	17
8	Asynchronní získání výsledků vyhledávání	21
9	Extrakce nemocí pro daný protein z UniProt	21
10	Načtení referenční sekvence proteinu	22
11	Výpočet procentuální shody sekvencí	23
12	Vyhledávání varianty v ClinVar	26
13	Získání popisu klasifikace varianty	27
14	Extrahování seznamu nemocí spojených s variantou	27
15	Uložení sekvencí do sessionStorage	29
16	Načtení sekvencí a vymazání sessionStorage	29
17	Formátování sekvenčních změn	31
18	Vykreslení obdélníku pomocí D3.js	32
19	Filtrace odpovídajících řádků	34
20	Zobrazení nebo skrytí sloupce na základě výběru	34
21	Procházení řádků a buněk tabulky	35
22	Vytvoření Blob objektu a stažení souboru	35

Seznam tabulek

1	Příklady přiřazení aminokyselin k jednotlivým kodonům [3]	7
2	Přehled zkratk aminokyselin [6]	8

Přílohy

1. Zdrojový kód aplikace – online přístupný na <https://github.com/jajik123/GenAnalyzer>
2. Odborný posudek – Ing. Ivana Burgetová, Ph.D., FIT VUT v Brně